Source code 1: Embedded code

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>


// Pin Definitions

const int ENA = 5;

const int IN1 = 6;

const int IN2 = 7;


const int LED1 = 13;

const int LED2 = 12;

const int LED3 = 11;


LiquidCrystal_I2C lcd(0x27, 16, 2);


void setup() {

  Serial.begin(9600);


  lcd.init();

  lcd.backlight();


  pinMode(ENA, OUTPUT);

  pinMode(IN1, OUTPUT);

  pinMode(IN2, OUTPUT);
```

```
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  pinMode(LED3, OUTPUT);

  lcd.setCursor(0,0);
  lcd.print("System Ready");
}

void loop() {
  if (Serial.available()) {

    String input = Serial.readStringUntil('\n'); // ✅ FIX
    input.trim(); // remove spaces/newline

    lcd.clear();

    if (input == "0") {
      analogWrite(ENA, 0);
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
      lcd.print("No Face");
      digitalWrite(LED1, LOW);
      digitalWrite(LED2, LOW);
      digitalWrite(LED3, LOW);
```

```
}
else if (input == "1") {
  analogWrite(ENA, 0);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  lcd.print("Happy");
  digitalWrite(LED1, HIGH);
  digitalWrite(LED2, LOW);
  digitalWrite(LED3, LOW);
}
else if (input == "2") {
  analogWrite(ENA, 160);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  lcd.print("Sad");
  digitalWrite(LED1, LOW);
  digitalWrite(LED2, HIGH);
  digitalWrite(LED3, LOW);
}
else if (input == "3") {
  analogWrite(ENA, 255);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  lcd.print("Angry");
```

```
      digitalWrite(LED1, LOW);
      digitalWrite(LED2, LOW);
      digitalWrite(LED3, HIGH);
    }
  }
}
```

Source code 2: python code

```
import tensorflow as tf
from keras.models import load_model
from time import sleep
from tensorflow.keras.preprocessing.image import img_to_array
from keras.preprocessing import image
import cv2
import numpy as np
#import pygame
import statistics
import serial
import time

COM_PORT="COM7"
baud_rate=9600
def connect_serial(port_name, baud_rate=9600):
    """
```

```
    Sets up the configuration and opens the port.
    Returns the serial object.
    """
    try:
        ser = serial.Serial(port_name, baud_rate, timeout=1)
        # Required for Windows/Arduino: wait for the hardware to initialize
        print(f"Connecting to {port_name}...")
        time.sleep(2)
        return ser
    except Exception as e:
        print(f"Configuration Error: {e}")

        return None

def send_data(serial_connection, message):
    """
    Uses an existing configuration to send data.
    """
    if serial_connection and serial_connection.is_open:
        try:
            # Format and send
            formatted_msg = f"{message}\n".encode('utf-8')
            serial_connection.write(formatted_msg)
            print(f"Data Sent: {message}")
```

```
        except Exception as e:
            print(f"Error sending data: {e}")
    else:
        print("Error: Serial port is not open.")

# --- Execution ---

# 1. Run Configuration once
my_device = connect_serial(COM_PORT, baud_rate)

# 2. Send data as many times as you want without waiting 2 seconds again
"""
if my_device:
    send_data(my_device, "LED_ON")
    send_data(my_device, "SET_SPEED_50")

    # 3. Always close the port when your program is completely finished
    my_device.close()
    print("Connection closed.")
"""

#pygame.mixer.init()
face_classifier = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
classifier =load_model('my_weights.h5')
```

```
#classifier =load_model('emotion_1.h5')
emotion_labels = ['Angry','Angry','Angry','Happy','Happy', 'Sad', 'Sad']
#emotions = ['neutral', 'anger', 'disgust', 'happy', 'sadness', 'surprise']
# prevents openCL usage and unnecessary logging messages

#cv2.ocl.setUseOpenCL(False)
"""
import os
import random

def get_random_file(directory):
    files = os.listdir(directory)
    random_file = random.choice(files)
    return os.path.join(directory, random_file)

def play_music(emotionPredictlist):
    if emotionPredictlist:

        mean= sum(emotionPredictlist) / len(emotionPredictlist)

        emotion=int(mean)

        song_directory="songs/{}".format(emotion)
```

```
        song=get_random_file(str(song_directory))


        while pygame.mixer.music.get_busy() != True:


            #pygame.mixer.music.load("song.mp3")
            #print "player playing"
            pygame.mixer.music.load(song)
            pygame.mixer.music.play()
"""
# Example usage
#directory = "/path/to/your/folder"
#random_file = get_random_file(directory)
#print("Randomly selected file:", random_file)


#cap = cv2.VideoCapture(0)
```